Evo Aura Billing Software

Codex Instructions: Reduce Long Files, Split Product/Supplier Pages, and Make Loading Smooth

Scope: Folder/file alteration and performance improvement only. No EXE packaging or installer work in this document.

Target: Windows 10 and later, PyQt6 application, SQLite local database, Git DB sync retained as code module.

1. Main instruction for Codex

The current project has working pages, but some files are too large. The biggest slow-loading files are Product and Supplier pages. Codex must split these files by responsibility, remove heavy cross-imports, cache pages instead of rebuilding them, and load detail/forms only when needed.

- Do not rewrite the whole application at once. Split in phases and keep the app running after every phase.
- Do not do EXE conversion, PyInstaller, installer, or Windows 7/8 compatibility work now.
- Keep PyQt6 as the final application framework. Do not import PyQt5 dashboard files directly into the PyQt6 app.
- Keep Git sync as a separate module. Do not place Git sync code inside UI page classes.
- Preserve existing class names when possible, or create compatibility wrapper files so old imports do not break immediately.

2. Existing files and what to do with each

Existing file	Current role	Codex alteration
evoaura_app(8).py	Main app: DB, auth, UI helpers, shell, routing, dashboard, Git sync import	Split into main.py, core/app_shell.py, core/db.py, core/migrations.py, core/auth_service.py, core/theme.py, core/ui_helpers.py, pages/auth/*.py, pages/dashboard_page.py
git_sync.py	Push local SQLite DB to private GitHub repo using QThread	Move to core/git_sync.py. Keep separate. Do not split now.
product_page(7).py	Huge Product Manager: theme, constants, DB, list, form, dialogs, stock, variants	Split into product controller, list, form, dialogs, widgets, repository, service, constants.
supplier_page(1).py	Supplier manager with list, detail, form, analytics and many product_page imports	Split into supplier controller, list, form, detail, dialogs, widgets, repository, service.
purchase_orders_page.py	Purchase invoice / stock-in page	Move to pages/inventory/purchase_invoices_page.py. Later split repository if heavy.
low_stock_page.py	Low stock / reorder control	Move to pages/inventory/reorder_control_page.py. Remove imports from product_page where possible.
customer_page.py	Customer center and customer tables	Move to pages/customers/customer_center_page.py. Move shared SQL to repository/migrations later.
credit_management_page.py	Due / collection control	Move to pages/customers/due_collection_page.py. Use customer repository/service later.
loyalty_page.py	Loyalty control center	Move to pages/customers/loyalty_page.py. Use customer repository/service later.
sidebar(2).py	Apple-style collapsible sidebar with navigation items	Move to core/sidebar.py. Optimize icon cache later.

Existing file	Current role	Codex alteration
dashboard(1).py	Old PyQt5 dashboard	Archive or convert useful UI ideas manually into pages/dashboard_page.py. Do not import directly.
app_branding.py	PyQt6 app icon helper	Move to core/app_branding.py.
input_behavior.py	PyQt6 global input guard	Move to core/input_behavior.py.
app_branding_qt5.py / input_behavior_qt5.py	PyQt5 helpers for old dashboard	Archive only. Do not use in final PyQt6 app.

3. Why Product and Supplier pages load slowly

Problem	Effect	Required fix
Many pages import common styles/helpers from product_page.py	Opening Supplier, Customer, Purchase, Low Stock, Credit, or Loyalty can force loading the huge Product file.	Move C, FIELD_SS, LABEL_SS, SEC_HDR_SS, _NO_ARROW, and combo delegate helper to core/theme.py and core/ui_helpers.py.
AppShell discards and rebuilds non-home pages on navigation	Clicking the same page again recreates Product/Supplier UI, which is slow.	Cache pages. Build once. Refresh lightly when reopened.
SupplierPage creates list, detail, and form immediately	Opening Supplier loads UI pieces the user may not use.	Create only list at first. Build form/detail only when Add/Edit/View is clicked.
ProductPage keeps list, form, dialogs, constants, DB logic in one huge file	Python import and page construction are heavy; Codex editing becomes difficult.	Split into controller/list/form/dialogs/widgets/repository/service/constants.
Large list queries and QTableWidget filling	UI freezes or feels laggy with many products/suppliers.	Use LIMIT/OFFSET, selected columns, no image load in list, setUpdatesEnabled(False) during table fill.
PyQt5 dashboard exists beside PyQt6 app	Mixing frameworks can cause confusion or runtime problems.	Do not import PyQt5 files into PyQt6. Convert ideas manually.

4. Final recommended folder structure

Keep the 41 main page/file plan, but add small support files for Product and Supplier performance. The 41 files are the application/page structure. Extra repository/service/widget files are performance support files and are acceptable.

```
evo_aura/
|-- main.py
|-- core/
|   |-- app_shell.py
|   |-- sidebar.py
|   |-- theme.py
|   |-- ui_helpers.py
|   |-- db.py
|   |-- migrations.py
|   |-- auth_service.py
|   |-- input_behavior.py
|   |-- app_branding.py
|   |-- git_sync.py
|   `-- product_constants.py
|-- repositories/
|   |-- product_repository.py
|   |-- supplier_repository.py
|   |-- customer_repository.py
|   |-- purchase_repository.py
|   `-- sales_repository.py
|-- services/
|   |-- product_service.py
|   |-- supplier_service.py
|   |-- customer_service.py
|   |-- purchase_service.py
|   `-- report_service.py
|-- pages/
|   |-- auth/
|   |-- inventory/
|   |   |-- product_page.py
|   |   |-- product_list.py
|   |   |-- product_form.py
|   |   |-- product_dialogs.py
|   |   |-- product_widgets.py
|   |   |-- supplier_page.py
|   |   |-- supplier_list.py
|   |   |-- supplier_form.py
|   |   |-- supplier_detail.py
|   |   |-- supplier_dialogs.py
|   |   |-- supplier_widgets.py
|   |   |-- purchase_invoices_page.py
|   |   `-- reorder_control_page.py
```

```
|   |-- customers/
|   |-- sales/
|   |-- finance/
|   |-- reports/
|   `-- settings/
|-- assets/icons/
`-- data/
```

5. First speed fix: remove product_page as a shared dependency

Many files should stop importing styles/helpers from product_page.py. This is the first high-impact change because it prevents Supplier/Customer/Purchase pages from importing the huge Product file just to use common colors or widgets.

Move these from product_page.py:

```
C
FIELD_SS
LABEL_SS
SEC_HDR_SS
_NO_ARROW
_apply_combo_delegate
```

New imports should become:

```
from core.theme import C, FIELD_SS, LABEL_SS, SEC_HDR_SS
from core.ui_helpers import NO_ARROW, apply_combo_delegate
```

Temporary compatibility wrapper for old imports:

```
# product_page.py at project root - temporary wrapper only
from pages.inventory.product_page import ProductPage
from repositories.product_repository import init_product_table
from core.theme import C, FIELD_SS, LABEL_SS, SEC_HDR_SS
from core.ui_helpers import NO_ARROW as _NO_ARROW, apply_combo_delegate as _apply_combo_delegate
```

After all imports are updated, root wrapper files can be removed later. Do not remove them during the first split if other files still import them.

6. Product page split plan

New file	Move/keep responsibility
pages/inventory/product_page.py	Small controller only: stack, open list, open add/edit form, handle saved/cancel. Target 80-150 lines.
pages/inventory/product_list.py	ProductListWidget: filter bar, table view, pagination, search, list refresh. Do not load images here.
pages/inventory/product_form.py	ProductFormWidget: 7-tab add/edit form. Build heavy tabs lazily when selected if possible.
pages/inventory/product_dialogs.py	Stock adjustment dialog, purchase stock dialog, manual stock dialog, invoice/product detail dialogs.
pages/inventory/product_widgets.py	Reusable product UI: buttons, panels, switches, charts, form field blocks.
repositories/product_repository.py	All SQL: init_product_table, fetch list, fetch one product, insert/update/delete, stock, variants, history.
services/product_service.py	Validation, margin calculation, barcode/SKU workflow, save transaction, stock business logic.
core/product_constants.py	Color groups, sub-category options, sizes, age categories, fabric/category constants.

ProductPage should become a thin controller:

```
class ProductPage(QWidget):
    def __init__(self, db_name, current_user="Admin", parent=None):
        super().__init__(parent)
        self.db_name = db_name
        self.current_user = current_user
        self.stack = QStackedWidget(self)
        self.list_widget = ProductListWidget(db_name)
        self.form_widget = None
        self.stack.addWidget(self.list_widget)
        self.list_widget.add_requested.connect(self.open_add)
        self.list_widget.edit_requested.connect(self.open_edit)

    def _ensure_form(self):
        if self.form_widget is None:
            self.form_widget = ProductFormWidget(self.db_name, self.current_user)
            self.form_widget.saved.connect(self.on_saved)
            self.form_widget.cancelled.connect(self.show_list)
            self.stack.addWidget(self.form_widget)

    def open_add(self):
        self._ensure_form()
        self.form_widget.load_for_add()
        self.stack.setCurrentWidget(self.form_widget)

    def open_edit(self, item_code):
        self._ensure_form()
        self.form_widget.load_for_edit(item_code)
        self.stack.setCurrentWidget(self.form_widget)

    def show_list(self):
        self.list_widget.refresh_light()
        self.stack.setCurrentWidget(self.list_widget)

    def on_saved(self, item_code):
        self.show_list()
```

Product list query should not load everything:

```
SELECT item_code, name, category, sub_category, brand, size, color,
       current_stock, mrp, selling_price, status
FROM products
WHERE is_deleted = 0
  AND (name LIKE ? OR item_code LIKE ? OR barcode LIKE ?)
ORDER BY name
LIMIT 100 OFFSET ?;
```

Do not load product image blobs in the list page. Load product image only when opening edit/detail.

7. Supplier page split plan

New file	Move/keep responsibility
pages/inventory/supplier_page.py	Small controller only: stack, list first, form/detail lazy.
pages/inventory/supplier_list.py	SupplierListWidget: KPIs, filters, supplier table, add/edit/view signals.
pages/inventory/supplier_form.py	SupplierFormWidget: add/edit supplier fields and validation.
pages/inventory/supplier_detail.py	SupplierDetailWidget: overview and lazy-loaded detail tabs.
pages/inventory/supplier_dialogs.py	Invoice detail, payment, product detail, confirmation dialogs.
pages/inventory/supplier_widgets.py	Reusable cards, table helpers, buttons, stat widgets.
repositories/supplier_repository.py	All SQL: init_supplier_tables, supplier list, detail, ledger, purchases, activity.
services/supplier_service.py	Supplier validation, KPI summary, balance calculations.

SupplierPage should lazy-build detail/form:

```
class SupplierPage(QWidget):
    def __init__(self, db_name, current_user="Admin", parent=None):
        super().__init__(parent)
        self.db_name = db_name
        self.current_user = current_user
        self.stack = QStackedWidget(self)
        self.list_page = SupplierListWidget(db_name)
        self.form_page = None
        self.detail_page = None
        self.stack.addWidget(self.list_page)
        self.list_page.add_requested.connect(self._add)
        self.list_page.edit_requested.connect(self._edit)
        self.list_page.view_requested.connect(self._view)

    def _ensure_form_page(self):
        if self.form_page is None:
            self.form_page = SupplierFormWidget(self.db_name, self.current_user)
            self.form_page.cancelled.connect(self._show_list)
            self.form_page.saved.connect(self._saved)
            self.stack.addWidget(self.form_page)

    def _ensure_detail_page(self):
        if self.detail_page is None:
            self.detail_page = SupplierDetailWidget(self.db_name)
            self.detail_page.back_requested.connect(self._show_list)
            self.detail_page.edit_requested.connect(self._edit)
            self.stack.addWidget(self.detail_page)
```

Supplier detail tabs should be lazy:

```
Open supplier detail
-> load only header + overview
-> when Purchase tab selected, query purchases
-> when Inventory tab selected, query supplier inventory
-> when Ledger tab selected, query ledger
-> when Activity tab selected, query activity log
```

8. AppShell route caching fix

The app should not discard and recreate non-home pages on every navigation. Product and Supplier are expensive pages. Build once, cache, and call refresh_light when returning.

Replace discard/rebuild behavior with cache behavior:

```
def _show_page(self, key: str):
    key = self._canonical_key(key)
    self._current_key = key

    icon, title = PAGE_META.get(key, ("doc", key.replace("_", " ").title()))
    self._page_title.setText(title)

    if key not in self._pages:
        page = self._build_page(key)
        self._pages[key] = page
        self._stack.addWidget(page)
    else:
        page = self._pages[key]
        if hasattr(page, "refresh_light"):
            page.refresh_light()

    self._stack.setCurrentWidget(self._pages[key])
    self._sidebar.set_active(key)
```

Only force-rebuild a page after major data changes if the page cannot refresh itself. Prefer refresh_light() over rebuild.

9. Database and table speed rules

Add indexes during migration. This gives immediate speed improvement for product search, barcode lookup, supplier detail, and reports.

```
CREATE INDEX IF NOT EXISTS idx_products_item_code ON products(item_code);
CREATE INDEX IF NOT EXISTS idx_products_barcode ON products(barcode);
CREATE INDEX IF NOT EXISTS idx_products_name ON products(name);
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category);
CREATE INDEX IF NOT EXISTS idx_products_brand ON products(brand);
CREATE INDEX IF NOT EXISTS idx_suppliers_code ON suppliers(code);
CREATE INDEX IF NOT EXISTS idx_suppliers_name ON suppliers(name);
CREATE INDEX IF NOT EXISTS idx_supplier_ledger_code ON supplier_ledger(supplier_code);
CREATE INDEX IF NOT EXISTS idx_purchase_orders_supplier ON purchase_orders(supplier_code);
CREATE INDEX IF NOT EXISTS idx_purchase_orders_date ON purchase_orders(purchase_date);
```

When filling QTableWidget:

```
table.setUpdatesEnabled(False)
table.blockSignals(True)
table.setSortingEnabled(False)
try:
    table.setRowCount(0)
    # fill rows here
finally:
    table.setSortingEnabled(True)
    table.blockSignals(False)
    table.setUpdatesEnabled(True)
```

For very large data later, replace QTableWidget with QTableView + QAbstractTableModel. For now, use LIMIT 100 and pagination/search.

10. Main app split plan

Move from evoaura_app(8).py	New file
QApplication startup, main window launch	main.py
AppShell, _ContentPage, page routing, sidebar integration	core/app_shell.py
C color dictionary, APP_SS, field styles	core/theme.py
F, L, gap, hline, Toast, SI, GB, LV	core/ui_helpers.py
DB class, connection helper	core/db.py
CREATE TABLE / ALTER TABLE / indexes	core/migrations.py
hash_pw, gen_recovery, verify_master, verify_admin_totp, TOTP QR helpers	core/auth_service.py
V_AskDB	pages/auth/ask_db_page.py
V_MasterAuth	pages/auth/master_auth_page.py
V_CompanySettings	pages/auth/company_setup_page.py
V_Signup	pages/auth/signup_page.py
V_QR	pages/auth/qr_setup_page.py
V_Login	pages/auth/login_page.py
V_OTP	pages/auth/otp_page.py
_DashboardPage	pages/dashboard_page.py

11. Import rules after split

After moving files into folders, imports should use package paths. Avoid root imports except temporary compatibility wrappers.

```
from core.sidebar import Sidebar
from core.theme import C, APP_SS
from core.ui_helpers import Toast, SI, GB, L, gap
from core.git_sync import GitSyncWorker, push_db_to_github

from pages.inventory.product_page import ProductPage
from pages.inventory.supplier_page import SupplierPage
from pages.inventory.purchase_invoices_page import PurchaseOrdersPage
from pages.inventory.reorder_control_page import LowStockPage
from pages.customers.customer_center_page import CustomerPage
from pages.customers.due_collection_page import CreditManagementPage
from pages.customers.loyalty_page import LoyaltyPage
```

12. Codex execution order

- Create folders: core, pages/auth, pages/inventory, pages/customers, pages/sales, pages/finance, pages/reports, pages/settings, repositories, services.
- Create __init__.py files in each package folder.
- Move app_branding.py, input_behavior.py, git_sync.py, and sidebar.py into core/. Update imports.
- Create core/theme.py and core/ui_helpers.py. Move common styles/helpers out of product_page.py.
- Update supplier, customer, purchase, low stock, credit, and loyalty pages so they no longer import common helpers from product_page.py.
- Split evoaura_app(8).py into core and auth page files. Keep app running.

- Fix AppShell page caching. Stop discarding pages during navigation.
- Split product_page(7).py into controller/list/form/dialogs/widgets/repository/service/constants.
- Split supplier_page(1).py into controller/list/form/detail/dialogs/widgets/repository/service.
- Make Product form and Supplier detail/form lazy-loaded.
- Add database indexes in core/migrations.py.
- Add LIMIT/OFFSET and no-image list loading in product and supplier list pages.
- Archive PyQt5 dashboard files; do not import them into PyQt6.

13. Acceptance checklist

- App opens without importing ProductPage when user only logs in and sees Dashboard.
- Opening Supplier page does not import heavy Product form code just for styles/helpers.
- Clicking Products, leaving, and returning does not rebuild the whole Product page.
- Product list opens quickly and shows first 100 rows or paginated rows.
- Product images are not loaded in the product list table.
- Supplier page opens with list only; detail and form are created only when needed.
- Supplier detail loads overview first; ledger/purchase/inventory/activity load only when tab is selected.
- No PyQt5 imports in main PyQt6 app path.
- Git sync remains in core/git_sync.py and uses QThread, not UI thread.
- No EXE conversion work is done in this alteration task.

14. Final instruction to Codex

Perform this as a safe refactor. Preserve behavior first, improve speed second. Use compatibility wrappers temporarily. After each phase, run the app and fix imports before continuing. Do not change visual design unless required by the split. The key performance goals are: no heavy cross-imports, cached pages, lazy-built forms/details, paginated database lists, and indexed search fields.